

How the T20 Power Rankings Are Calculated

A power ranking answers a different question from the league table. The table asks “who has earned the most points?” The power ranking asks “who is playing the best cricket right now?” — so a team can sit fourth in the table and still top this list.

The founding rule: the maths decides the order, the AI does not. Every position is computed by fixed formulas from match data. The language model is only allowed to write the sentence explaining a team’s position — it never sees, chooses, or influences the rank. Every ranking is reproducible, and every place is answerable with a number.

Part 1 — In plain English

Think of it as a school report card. Instead of one exam, each team is graded on nine subjects. Every subject is marked out of 100, some subjects count more than others, and the marks are combined into one final score. Sort by that score, and you have the rankings.

#	What we measure	What it tells you	Counts for
1	Win %	How often they win. The foundation.	30%
2	Death overs (16–20)	The closing overs, where T20 games are decided: their hitting versus their bowling.	18%
3	Powerplay (overs 1–6)	Command of the opening burst — runs scored there minus runs conceded.	16%
4	Rolling Net Run Rate	Whether they out-score opponents, over the last 5 games only, so one early thrashing doesn’t flatter them all season.	9%
5	Momentum (form)	Are they hot right now? The latest match counts most.	8%
6	Home / away	Teams that only win at home are flattered; travelling well counts.	8%
7	Key players	A side missing its stars is not the same side, whatever last week’s results say.	5%
8	Win quality	How they win — a 60-run rout versus a last-ball escape.	3%
9	Strength of schedule	Whether the opponents beaten were any good.	3%

These weights were measured, not chosen. Rather than arguing about how much the powerplay “should” be worth, we built 300 simulated seasons in which each team’s true strength was known in advance, then searched for the weights that best recovered that true order. We tuned on two-thirds of those seasons and scored on the third we had not touched. The fitted weights rank teams noticeably better than our original hand-picked ones (rank correlation with true strength improved from 0.69 to 0.76).

The two subjects worth only 3%. Honesty matters more than a tidy table. That same test showed win quality carried almost no signal as we currently compute it, and strength of schedule actively worked backwards in a full round-robin — where everyone plays everyone, your opponents’ average win rate is essentially the mirror of your own. Both were cut to a token weight rather than quietly dropped, and both need rebuilding before they earn it back.

Why momentum isn’t simply “the last five”. Recent matches count far more than old ones, fading smoothly rather than stopping dead at a cut-off. In practice the most recent match carries about 35% of this grade, and the last five together about 88%.

The score and the arrows. The nine graded subjects combine into a score out of 100; teams typically land between about 30 and 70. Nobody scores 100 — that would mean leading every category at once. The ▲/▼ arrows show movement against the previous published ranking: ▲2 means two places gained.

Nothing here is opinion. Most published power rankings are a writer’s judgement call. This one is arithmetic: run the same matches through it twice and you get the same table, every time.

Part 2 — The technical method

Pipeline. data → collect() → nine metrics → normalise to [0,1] → weighted sum × 100 → sort. Weights and bounds live in weights.config.json, never in code. Implementation: src/engine.js.

Notation. For team t: W/P = wins/played; RF, OF = runs scored and overs faced; RA, OA = runs and overs conceded. Every metric derives from raw per-innings data — nothing is hand-entered.

The nine components

1. Win percentage — W / P

2. Death-overs net (overs 16–20)

$\text{net} = \text{deathBatSR} - \text{deathBowEcon} \times 16.67$

The factor $100/6 = 16.67$ converts bowling economy (runs/over) onto the batting strike-rate scale (runs per 100 balls), making the two dimensionally comparable.

3. Powerplay dominance (overs 1–6, i.e. 6 overs per innings per match)

$\text{PP} = (\text{ppRunsFor} / 6P) - (\text{ppRunsAgainst} / 6P)$ [runs per over]

4. Rolling NRR (last 5 matches)

$\text{rNRR} = (\sum \text{RF} / \sum \text{OF}) - (\sum \text{RA} / \sum \text{OA})$

Also reported: trend = rolling – season, positive meaning improving.

5. Momentum — exponentially weighted update over every match played, seeded at 0.5:

$m_t = \alpha \cdot (\text{win} ? 1 : 0) + (1 - \alpha) \cdot m_{t-1}$, $\alpha = 0.35$

Match weights decay 0.350, 0.228, 0.148, 0.096, 0.063... — a half-life of ≈ 1.6 matches.

6. Home/away — $0.6 \cdot \text{awayWin\%} + 0.4 \cdot \text{homeWin\%}$, deliberately over-weighting away form.

7. Key-player availability — $\text{starsAvailable} / \text{squadStars}$, taken as of the most recent match.

8. Win quality (mean over wins only, each clamped to 0–100)

batting first : $(\text{own} - \text{opp}) / \text{own} \times 100$
chasing : $(120 - \text{balls used}) / 120 \times 100$

9. Strength of schedule — mean current win% of all opponents faced (repeat fixtures counted each time).

Normalisation and the final score

Each raw metric maps onto $[0,1]$ via $\text{norm}(x) = \text{clamp}((x - \text{min})/(\text{max} - \text{min}))$. Ratios (1, 6, 7, 9) are already 0–1; the rest use configured bounds — death net ± 60 , powerplay ± 3.00 rpo, rNRR ± 2.00 , win quality ± 100 . Then:

$\text{score} = 100 \times \sum (w_k \cdot n_k)$

weights 0.30, 0.18, 0.16, 0.09, 0.08, 0.08, 0.05, 0.03, 0.03, summing to exactly 1.00. Ties break on win% then rolling NRR. Delta = previous rank – current rank.

Fitting method. Weights maximise $\text{corr}(\text{score}, \text{latent strength})$ subject to $w \geq 0$, $\sum w = 1$, over 300 seeded synthetic seasons, trained on 66% and evaluated on the held-out 34% (held-out Spearman 0.694 $\rightarrow$ 0.760; correct #1 46% $\rightarrow$ 50%). The unconstrained optimum (0.892) was rejected: it loaded ~75% of the table onto the two phase metrics and dropped Win% to 0.05, which would let a team win every match and not rank first.

Optional metrics (off by default)

Five extras can be enabled per session: expected wins (Pythagorean $\text{RF}^2/(\text{RF}^2 + \text{RA}^2)$), toss independence (win% when losing the toss), chase/set versatility, top-4 consistency, and bowling spread (inverse share of wickets taken by the top two bowlers). Enabling them pushes total weight above 1.0, so scores scale up while the ordering stays valid.

Known limitations — stated deliberately

- Ground truth is synthetic. Weights are fitted against a generator, not real results, so the phase metrics are probably over-credited. Re-fitting on a full season of live data is the priority; the method transfers unchanged.
- NRR convention. ICC rules charge a side bowled out with its full quota of overs; this engine divides by overs actually faced, slightly flattering teams dismissed early.
- Win quality (3%) measured $r \approx +0.01$ — no signal as computed. It averages only over wins, and its batting-first and chasing scales are not calibrated to each other.
- Strength of schedule (3%) measured $r = -1.00$ in a complete round robin and is added positively, so it currently penalises the best teams. The sign needs fixing, or the metric restricting to unbalanced schedules.
- T20-shaped. 120 balls and a 6-over powerplay are assumed; rain-shortened games and other formats are not modelled.
- Key players is a supplied figure, not yet a live ICC top-30 feed.

Computed by `src/engine.js`; weights tunable in `weights.config.json` without code changes. Full fitting write-up in `parameters.md`. Every generated blurb is fact-checked against these numbers and logged with its sources for editorial review.